12. BUILD & DEPLOYMENT

The Build and Deployment phase is responsible for compiling the project, managing dependencies, executing automated test cases, and deploying the project source code to a version control repository. This phase ensures that the automation framework can be executed consistently across different environments and can be shared with other team members through GitHub.

For this project, Maven is used as the build automation tool, TestNG is used for test execution, and GitHub is used for source code management and project deployment.

12.1 Maven Build Process

Apache Maven is a build automation and dependency management tool used to manage project libraries, compile source code, execute test cases, and generate reports.

In this project, Maven plays a crucial role in simplifying dependency management and automating the build process.

Purpose of Maven

The primary purposes of Maven in this project are:

- Managing project dependencies
- Compiling source code
- Executing automated test cases
- Generating test reports
- Packaging project artifacts
- Maintaining project consistency

Maven Project Structure

The project follows a standard Maven directory structure:

```
AUTOMATION-04-Web-Application-Security-Testing-DVWA-ZAP
│
├── src/test/java
├── src/main/java
├── src/test/resources
├── reports
├── screenshots
├── test-output
├── pom.xml
└── testng.xml
```

This structure improves project organization and maintainability.

Dependency Management

All required libraries are managed through the **pom.xml** file.

Key dependencies used in the project include:

- Selenium WebDriver
- TestNG
- WebDriverManager
- Extent Reports
- OWASP ZAP API Client

Maven automatically downloads and manages these dependencies from the Maven Central Repository.

Build Lifecycle

The Maven Build Lifecycle consists of multiple phases:

Validate

Checks whether the project structure and configuration are correct.

Compile

Compiles Java source code into executable class files.

Test

Executes TestNG test cases.

Package

Packages the project into a deployable artifact.

Verify

Verifies build quality and execution results.

Install

Installs the build artifact into the local Maven repository.

Build Execution

The project build is executed using the following command:

```
mvn clean test
```

Command Explanation

mvn clean

Removes previous build files and execution results.

mvn test

Compiles source code and executes all TestNG test cases.

Build Output

After successful execution:

- Test cases are executed.
- TestNG Reports are generated.
- Extent Reports are generated.
- Screenshots are captured.
- Security testing results are recorded.

Build Verification

A successful build indicates:

- No compilation errors.
- All dependencies resolved successfully.
- Test execution completed.
- Reports generated correctly.

The Maven build process ensures that the framework remains stable, reusable, and easy to execute.

12.2 Project Execution Steps

Project execution involves running automated functional and security test cases against DVWA and generating execution reports.

The following steps were performed during project execution.

Step 1: Start XAMPP Services

Open XAMPP Control Panel.

Start:

- Apache Server
- MySQL Server

Both services must display a running status.

Step 2: Launch DVWA Application

Open browser and access:

```
http://localhost/DVWA
```

Verify that the DVWA login page loads successfully.

Step 3: Verify Database Connectivity

Open:

```
http://localhost/phpmyadmin
```

Verify that:

```
dvwa
```

database exists and is accessible.

Step 4: Start OWASP ZAP

Launch OWASP ZAP.

Verify:

- ZAP application starts successfully.
- API access is enabled.
- Proxy settings are configured correctly.

Default configuration:

```
Host : localhost
Port : 8080
```

Step 5: Open Eclipse IDE

Launch Eclipse IDE.

Open the project:

```
AUTOMATION-04-Web-Application-Security-Testing-DVWA-ZAP
```

Verify that:

- No build errors exist.
- Maven dependencies are loaded successfully.

Step 6: Execute Login Test

Run Login Test Class.

Activities performed:

- Launch Chrome Browser.
- Open DVWA Login Page.
- Enter credentials.
- Verify successful login.

Expected Result:

User should navigate to DVWA Home Page.

Step 7: Execute SQL Injection Test

Run SQL Injection Test Class.

Activities performed:

- Navigate to SQL Injection Module.
- Submit malicious SQL payloads.
- Analyze application responses.

Expected Result:

Application should reject malicious payloads.

Actual Result:

SQL Injection vulnerability identified.

Step 8: Execute XSS Test

Run XSS Test Class.

Activities performed:

- Navigate to XSS Module.
- Submit malicious JavaScript payloads.
- Observe browser behavior.

Expected Result:

Scripts should be sanitized.

Actual Result:

XSS vulnerability detected.

Step 9: Execute OWASP ZAP Security Scan

Run ZAP Security Scan Module.

Activities performed:

Spider Scan

- Discover application pages.
- Identify URLs and parameters.

Active Scan

- Test discovered pages for vulnerabilities.
- Generate security alerts.

Expected Result:

Security vulnerabilities should be detected successfully.

Step 10: Execute Complete Test Suite

Run:

```
testng.xml
```

This executes:

- Login Tests
- SQL Injection Tests
- XSS Tests
- OWASP ZAP Security Scans

in a single execution cycle.

Step 11: Generate Reports

After execution:

TestNG Report

Generated in:

```
test-output
```

Extent Report

Generated in:

```
reports
```

OWASP ZAP Report

Generated through:

```
OWASP ZAP Report Export
```

All reports are reviewed and analyzed.

Project Execution Workflow

```
Start XAMPP
        ↓
Launch DVWA
        ↓
Start OWASP ZAP
        ↓
Open Eclipse Project
        ↓
Execute Login Tests
        ↓
Execute SQL Injection Tests
        ↓
Execute XSS Tests
        ↓
Execute OWASP ZAP Scans
        ↓
Run TestNG Suite
        ↓
Generate Reports
        ↓
Analyze Results
```

This workflow ensures complete execution of all testing activities.

12.3 GitHub Deployment

GitHub is used as the version control and source code management platform for the project.

Deploying the project to GitHub provides:

- Source code backup
- Version control
- Collaboration support
- Portfolio showcase
- Easy project sharing

Purpose of GitHub Deployment

GitHub deployment helps:

- Maintain project history.
- Track code changes.
- Store project documentation.
- Share project with recruiters and reviewers.
- Support future enhancements.

Git Initialization

Open Git Bash inside project directory.

Initialize repository:

```
git init
```

This creates a local Git repository.

Add Project Files

Add all project files:

```
git add .
```

This stages files for commit.

Create Initial Commit

Create first commit:

```
git commit -m "Initial Project Commit"
```

The commit stores the current project version.

Create GitHub Repository

Login to GitHub.

Create repository:

```
AUTOMATION-04-Web-Application-Security-Testing-DVWA-ZAP
```

Configure:

- Repository Name
- Description
- Visibility (Public/Private)

Connect Local Repository to GitHub

Configure remote repository:

```
git remote add origin <repository-url>
```

Example:

```
git remote add origin https://github.com/username/AUTOMATION-04-Web-Application-Security-Testing-DVWA-ZAP.git
```

Push Project to GitHub

Push source code:

```
git branch -M main
git push -u origin main
```

The project is uploaded successfully.

Repository Contents

The GitHub repository contains:

```
Project Source Code
Automation Scripts
Page Object Classes
Configuration Files
pom.xml
testng.xml
Screenshots
Reports
Documentation
README.md
```

GitHub Benefits

GitHub provides several advantages:

Version Control

Tracks every code modification.

Backup

Maintains a secure copy of project files.

Collaboration

Allows multiple developers to work together.

Portfolio Building

Showcases automation and security testing skills.

Continuous Improvement

Supports future enhancements and maintenance.

Build & Deployment Summary

The Build and Deployment phase ensured successful compilation, execution, and deployment of the DVWA Security Testing Automation Framework. Maven simplified dependency management and automated the build process, while TestNG executed the test suite and generated execution reports. GitHub deployment provided version control, project backup, and a professional platform for sharing the project. Together, these activities ensured that the project was fully executable, maintainable, and ready for future development and demonstration purposes.